Guía para la implementación de los **Temas Técnicos**

En la **Etapa V** del proyecto, cada equipo deberá investigar e implementar mecanismos avanzados del motor **SQL Server Express** que permitan mejorar la robustez, confiabilidad y mantenibilidad de la base de datos del sistema de ventas desarrollado. Los temas obligatorios son:

1. **procedimientos y funciones almacenadas,**
2. **manejo de transacciones, y**
3. **triggers de auditoría**.
4. Seguridad
5. Optimización de consultas

Para cada tema se solicita presentar:

1. Definición y objetivo.
2. Ventajas y aplicaciones.
3. Implementación en SQL Server Express.
4. Caso concreto aplicado al sistema de ventas del equipo.
5. Script de demostración y resultados obtenidos.

1. Procedimientos y Funciones Almacenadas

¿Qué son?

Los procedimientos almacenados (*Stored Procedures*) son programas SQL que se guardan dentro de la base de datos y pueden ejecutar múltiples operaciones. Las funciones almacenadas (*Functions*) son rutinas que reciben parámetros y devuelven un valor o una tabla.

¿Por qué utilizarlos?

- Reutilizan lógica de negocio.
- Reducen código repetido.
- Mejoran la mantenibilidad.
- Centralizan reglas de negocio.
- Facilitan la seguridad sobre los datos.

Caso de aplicación de ejemplo

Función: Calcular descuento para clientes frecuentes

Problema de negocio: El sistema debe otorgar descuentos según el monto acumulado de compras de un cliente.

Función propuesta:

```
dbo.fn_CalcularDescuentoCliente(@cliente_id)
```

Lógica:

- Más de $500.000 acumulados ⟶ 10% descuento.
- Más de $200.000 acumulados ⟶ 5% descuento.
- Caso contrario ⟶ 0%.

Uso:

```
SELECT
    nombre,
    dbo.fn_CalcularDescuentoCliente(cliente_id) AS descuento
FROM cliente;
```

Procedimiento: Registrar una venta

Problema de negocio: Registrar una venta completa sin que el usuario deba ejecutar múltiples consultas.

Procedimiento propuesto:

```
1   dbo.sp_RegistrarVentaCompleta
```

Tareas que realiza:

1. Registrar encabezado de venta.
2. Insertar detalle de venta.
3. Guardar precio histórico.
4. Actualizar stock.
5. Aplicar descuentos.
6. Confirmar la operación.

Beneficio:

Toda la lógica de ventas queda centralizada en un único componente reutilizable.

2. Manejo de Transacciones

¿Qué son?

Una transacción es un conjunto de operaciones SQL que se ejecutan como una única unidad lógica de trabajo. Si una operación falla, todas las anteriores se deshacen. Esto garantiza las propiedades **ACID**: Atomicidad, Consistencia, Aislamiento y Durabilidad.

¿Por qué utilizarlas?

Porque una venta involucra múltiples tablas:

- Venta
- Detalle_Venta
- Producto
- Stock

No debe quedar información incompleta.

Caso de aplicación de ejemplo

Venta con control de stock

Escenario:

Un cliente compra 5 unidades de un producto.

La operación debe:

1. Crear la venta.
2. Crear el detalle.
3. Descontar stock.

Si no existe stock suficiente:

- No se registra la venta.
- No se registra el detalle.
- No se modifica stock.

Ejemplo conceptual

```
BEGIN TRANSACTION;

-- Registrar venta

-- Registrar detalle

-- Descontar stock

IF @stock_actual < @cantidad
  BEGIN
    ROLLBACK TRANSACTION;
  END
  ELSE
  BEGIN
    COMMIT TRANSACTION;
  END
```

Demostración sugerida

Realizar dos pruebas:

Correcta

Producto con stock suficiente.

Resultado:

- Venta registrada.
- Stock actualizado.

Incorrecta

Producto con stock insuficiente.

Resultado:

- Se ejecuta ROLLBACK.
- No queda ningún registro parcial.

De esta forma se demuestra la propiedad de **atomicidad**.

3. Triggers de Auditoría

¿Qué son?

Los triggers son procedimientos que se ejecutan automáticamente cuando ocurre un evento sobre una tabla:

- INSERT
- UPDATE
- DELETE

Se utilizan frecuentemente para auditorías y trazabilidad.

¿Por qué utilizarlos?

Permiten responder preguntas como:

- ¿Quién modificó un precio?
- ¿Cuándo se eliminó un producto?
- ¿Qué valor tenía antes de la modificación?

Caso de aplicación de ejemplo

Auditoría de productos

Supongamos que un usuario modifica el precio de un producto.

Tabla principal:

```
1  producto
```

Tabla de auditoría:

```
1  auditoria_producto
```

Información a registrar

- Fecha y hora.
- Usuario.
- Producto modificado.
- Precio anterior.
- Precio nuevo.
- Tipo de operación.

Ejemplo práctico

Antes:

Producto	Precio
Mouse	15000

Actualización:

```
1  UPDATE producto
2  SET precio = 18000
3  WHERE producto_id = 1;
```

Registro generado automáticamente:

Fecha	Producto	Precio Anterior	Precio Nuevo
08/10/2026	Mouse	15000	18000

Demostración sugerida

1. Modificar un precio.
2. Consultar la tabla de auditoría.
3. Verificar que se registró el cambio automáticamente.

Esto demuestra la capacidad de trazabilidad y control de cambios del sistema.

Tema **Seguridad**: Manejo de permisos a nivel de usuarios y roles en SQL Server

¿Qué es?

La seguridad en una base de datos permite controlar qué usuarios pueden acceder a la información y qué operaciones pueden realizar. SQL Server permite administrar permisos de manera individual (usuarios) o agrupándolos mediante roles. Esto facilita proteger la información y aplicar el principio de mínimo privilegio.

¿Por qué utilizar permisos y roles?

- Protegen la información sensible del sistema.
- Evitan modificaciones accidentales o no autorizadas.
- Permiten asignar responsabilidades según el perfil del usuario.
- Simplifican la administración de accesos mediante roles reutilizables.
- Mejoran la seguridad y mantenibilidad de la base de datos.

Caso de aplicación de ejemplo

Escenario

Supongamos que en un sistema de ventas existe un procedimiento almacenado denominado:

```
1    dbo.sp_RegistrarVentaSimple
```

Este procedimiento registra una venta en la tabla Venta.

Se desea demostrar:

1. Permisos asignados directamente a usuarios.
2. Permisos asignados mediante roles.
3. Diferencia entre ejecutar una sentencia SQL directamente y utilizar un procedimiento almacenado.

El ejemplo utiliza una tabla y procedimiento simplificados. Cada equipo deberá adaptarlo a su propio modelo de ventas (productos, clientes, servicios, etc.).

Parte 1 - Permisos a nivel de usuarios

Creación de usuarios

Se crean dos inicios de sesión y sus respectivos usuarios.

```
CREATE LOGIN usuario_admin
WITH PASSWORD = 'Admin123*';

CREATE LOGIN usuario_lector
WITH PASSWORD = 'Lector123*';
GO

USE SistemaVentas;
GO

CREATE USER usuario_admin FOR LOGIN usuario_admin;
CREATE USER usuario_lector FOR LOGIN usuario_lector;
GO
```

Asignación de permisos

Usuario administrador

Se agrega al rol de administrador de la base.

```
ALTER ROLE db_owner
ADD MEMBER usuario_admin;
GO
```

Usuario con permisos limitados

Se asigna únicamente permiso de lectura.

```
GRANT SELECT ON dbo.Venta TO usuario_lector;
GO
```

Permiso de ejecución sobre procedimiento almacenado

Aunque el usuario solo puede consultar datos, se le permite ejecutar un procedimiento específico.

```
GRANT EXECUTE
ON dbo.sp_RegistrarVentaSimple
TO usuario_lector;
GO
```

Prueba 1: INSERT directo sobre la tabla

Con usuario administrador

```
INSERT INTO Venta(fecha_venta,total)
VALUES(GETDATE(),15000);
```

Resultado esperado: operación exitosa.

Con usuario lector

```
INSERT INTO Venta(fecha_venta,total)
VALUES(GETDATE(),15000);
```

Resultado esperado: SQL Server rechaza la operación por falta de permisos de inserción.

Prueba 2: INSERT mediante procedimiento almacenado

Usuario lector

```
EXEC dbo.sp_RegistrarVentaSimple
    @Total = 15000;
```

Resultado esperado: la venta se registra correctamente porque el usuario posee permiso de ejecución sobre el procedimiento.

Entonces...

El usuario no puede modificar la tabla directamente, pero sí puede realizar la operación autorizada a través del procedimiento almacenado, conservando el control de la lógica de negocio.

Parte 2 - Permisos a nivel de roles

Creación de usuarios

```
CREATE LOGIN usuario_con_rol
WITH PASSWORD = 'Rol123*';

 CREATE LOGIN usuario_sin_rol
WITH PASSWORD = 'Rol456*';
GO

CREATE USER usuario_con_rol
FOR LOGIN usuario_con_rol;

CREATE USER usuario_sin_rol
```

```
12    FOR LOGIN usuario_sin_rol;
13    GO
```

Creación de un rol personalizado

Se crea un rol para consultar productos.

```
1     CREATE ROLE rol_consulta_productos;
2     GO
```

Asignación de permisos al rol

```
1     GRANT SELECT
2     ON dbo.Producto
3     TO rol_consulta_productos;
4     GO
```

Incorporación de usuario al rol

```
1     ALTER ROLE rol_consulta_productos
2     ADD MEMBER usuario_con_rol;
3     GO
```

Verificación de comportamiento

Usuario con rol

```
1     SELECT *
2     FROM Producto;
```

Resultado esperado: consulta exitosa.

Usuario sin rol

```
1     SELECT *
2     FROM Producto;
```

Resultado esperado: SQL Server deniega el acceso por falta de permisos.

Demostración sugerida

Realizar las siguientes pruebas:

Prueba	Usuario	Resultado esperado
SELECT sobre Producto	usuario_con_rol	Correcto
SELECT sobre Producto	usuario_sin_rol	Acceso denegado
INSERT directo sobre Venta	usuario_admin	Correcto
INSERT directo sobre Venta	usuario_lector	Acceso denegado
EXEC procedimiento	usuario_lector	Correcto

Tema: Optimización de consultas a través de índices

¿Qué es?

Los índices son estructuras que SQL Server utiliza para localizar información de forma más eficiente. Su funcionamiento es similar al índice de un libro: en lugar de recorrer todas las páginas para encontrar un tema, permite acceder directamente a la información buscada.

Cuando una tabla contiene una gran cantidad de registros, las consultas pueden requerir recorrer toda la tabla (Table Scan), generando tiempos de respuesta elevados. La implementación adecuada de índices permite reducir significativamente el volumen de datos analizados durante una consulta.

¿Por qué utilizar índices?

- Mejoran el rendimiento de las consultas de búsqueda.
- Reducen los tiempos de respuesta en tablas con grandes volúmenes de datos.
- Disminuyen la cantidad de lecturas que realiza el motor de base de datos.
- Permiten optimizar filtros por fechas, códigos, clientes, productos, entre otros.
- Facilitan la escalabilidad de sistemas con grandes cantidades de información.

> Sin embargo, los índices también consumen espacio y pueden afectar levemente el rendimiento de las operaciones INSERT, UPDATE y DELETE, ya que deben mantenerse actualizados.

Caso de aplicación de ejemplo

Escenario

Supongamos que un sistema de ventas registra operaciones diariamente en una tabla denominada:

```
1    VentaHistorica
```

La empresa desea analizar las ventas realizadas durante determinados períodos de tiempo.

Para evaluar el impacto de los índices se realizará una prueba sobre una tabla con más de un millón de registros.

> Este ejemplo utiliza una tabla simplificada. Cada grupo deberá adaptar la solución a una tabla de su propio modelo de ventas.

Parte 1 - Creación de la tabla de pruebas

```
CREATE TABLE VentaHistorica
(
    id_venta BIGINT IDENTITY(1,1),
    fecha_venta DATE NOT NULL,
    importe DECIMAL(12,2) NOT NULL
);
GO
```

Parte 2 - Carga masiva de datos

Se genera información simulada para poblar la tabla con un volumen significativo de registros.

```
SET NOCOUNT ON;
GO

WITH Numeros AS
(
    SELECT TOP (1000000)
           ROW_NUMBER() OVER(ORDER BY (SELECT NULL)) AS n
    FROM sys.objects a
    CROSS JOIN sys.objects b
    CROSS JOIN sys.objects c
)
INSERT INTO VentaHistorica(fecha_venta, importe)
SELECT
    DATEADD(DAY, ABS(CHECKSUM(NEWID())) % 1825,
'2020-01-01'),
    RAND(CHECKSUM(NEWID())) * 10000
FROM Numeros;
GO
```

Verificar la cantidad de registros:

```
SELECT COUNT(*) AS CantidadRegistros
FROM VentaHistorica;
```

Parte 3 - Consulta sin índices

Activar las estadísticas de rendimiento:

```
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO
```

Consulta de ejemplo:

```
SELECT
    id_venta,
    fecha_venta,
    importe
FROM VentaHistorica
WHERE fecha_venta BETWEEN
      '2023-01-01'
  AND '2023-03-31';
```

Observaciones a registrar

Plan de ejecución

Analizar si SQL Server utiliza:

```
Table Scan
```

o

```
Clustered Index Scan
```

según la estructura existente.

Métricas

Registrar:

- Tiempo de CPU.
- Tiempo total de ejecución.
- Cantidad de lecturas lógicas.
- Porcentaje de costo de la consulta.
- Captura del plan de ejecución.

Ejemplo de documentación:

Medición	Valor
Tiempo CPU	XX ms
Tiempo total	XX ms
Lecturas lógicas	XX
Operación principal	Table Scan

Los valores variarán según el hardware utilizado.

Parte 4 - Crear índice agrupado sobre la fecha

Crear un índice sobre la columna utilizada en el filtro.

```
CREATE CLUSTERED INDEX IX_VentaHistorica_Fecha
ON VentaHistorica(fecha_venta);
GO
```

Repetir la consulta

```
SELECT
    id_venta,
    fecha_venta,
    importe
FROM VentaHistorica
WHERE fecha_venta BETWEEN
      '2023-01-01'
  AND '2023-03-31';
```

Observaciones a registrar

Analizar si el optimizador cambia su estrategia de acceso:

```
Clustered Index Seek
```

o alguna combinación de:

```
Seek + Lookup
```

Registrar nuevamente:

- Tiempo de ejecución.
- Tiempo de CPU.
- Lecturas lógicas.
- Plan de ejecución.

Parte 5 - Eliminar el índice

```
DROP INDEX IX_VentaHistorica_Fecha
ON VentaHistorica;
GO
```

Parte 6 - Crear un índice que cubra la consulta

En este segundo escenario se crea un índice que permita resolver la consulta accediendo a menor cantidad de datos.

```
CREATE NONCLUSTERED INDEX IX_VentaHistorica_Fecha_Cobertura
ON VentaHistorica(fecha_venta)
INCLUDE (importe);
GO
```

Dependiendo del diseño elegido por cada grupo, podrían incluirse otras columnas frecuentemente consultadas.

Repetir la consulta

```
SELECT
    fecha_venta,
    importe
FROM VentaHistorica
WHERE fecha_venta BETWEEN
      '2023-01-01'
  AND '2023-03-31';
```

Observaciones a registrar

Analizar si SQL Server puede resolver la consulta utilizando únicamente el índice.

Verificar la aparición de operadores como:

```
Index Seek
```

sin necesidad de acceder a la tabla completa.

Registrar:

- Tiempo de CPU.
- Tiempo total.
- Lecturas lógicas.
- Plan de ejecución.

Comparación sugerida

Se puede completar una tabla similar a la siguiente:

Escenario	Tiempo (ms)	Lecturas Lógicas	Operación Principal
Sin índice	XX	XX	Table Scan
Índice sobre fecha	XX	XX	Index Seek
Índice con cobertura	XX	XX	Index Seek

Resultados esperados

En general debería observarse que:

1. La consulta sin índices requiere analizar una gran cantidad de registros.
2. El índice sobre la fecha reduce significativamente el trabajo realizado por el motor.
3. El índice con cobertura disminuye aún más las lecturas necesarias para responder la consulta.
4. Los planes de ejecución muestran un cambio desde operaciones de escaneo completo hacia búsquedas más eficientes.